

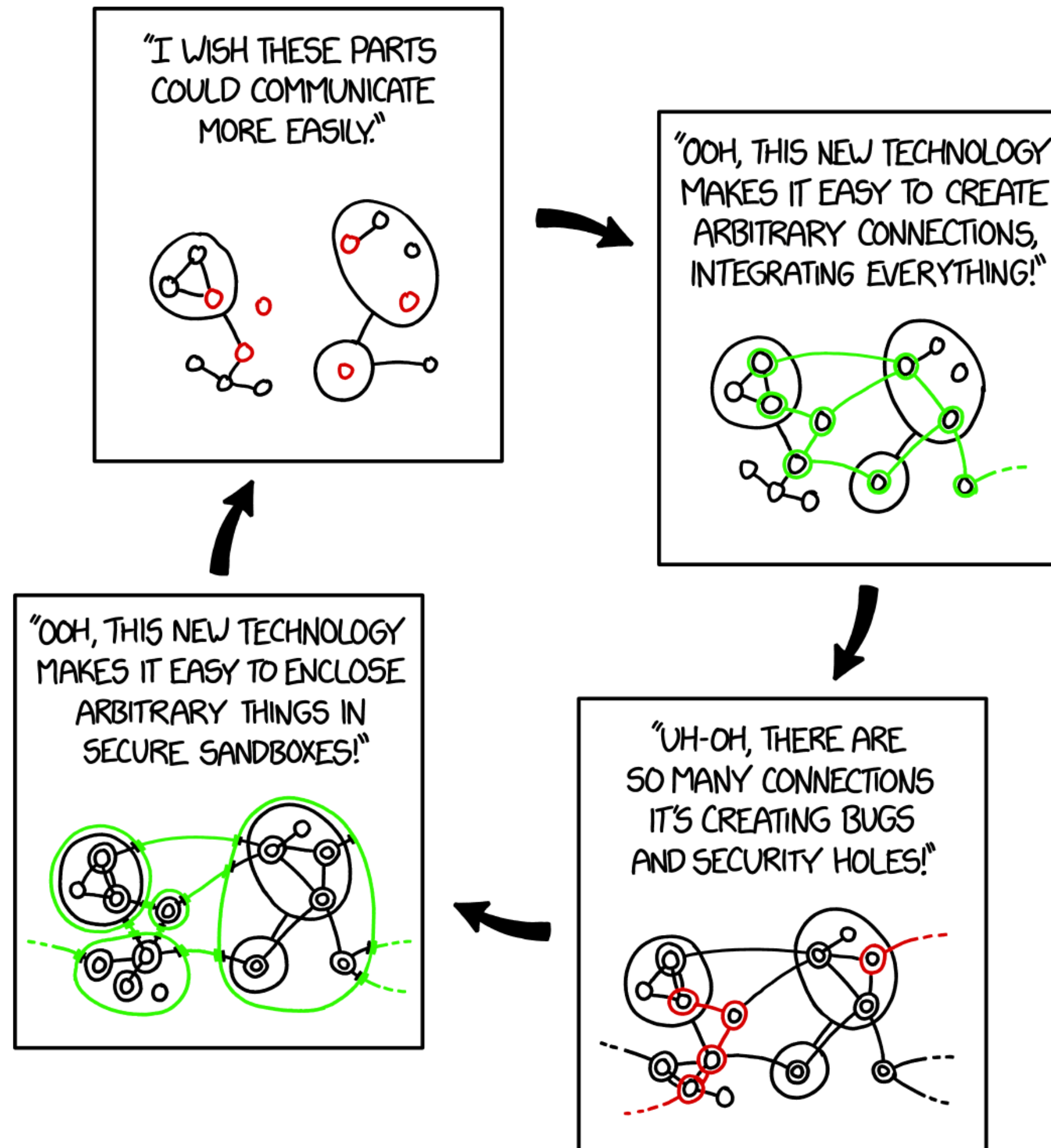
ISE 331: Computer Security Fundamentals

Spring 2026

Sandboxing



Amir Rahmati, Xinrui Yu



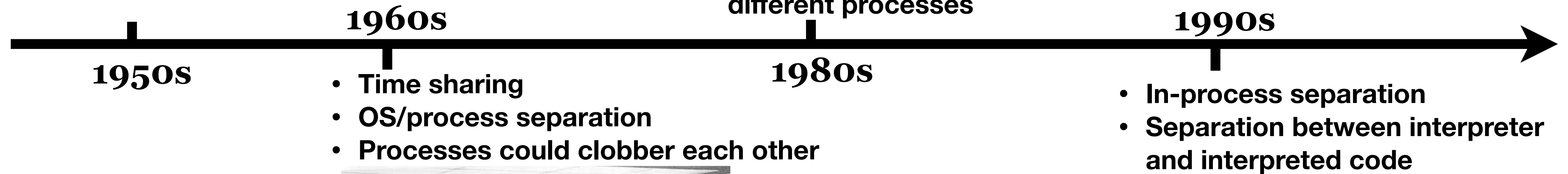


- Bare metal
- One process at a time
- Omnipotent processes

Quick History

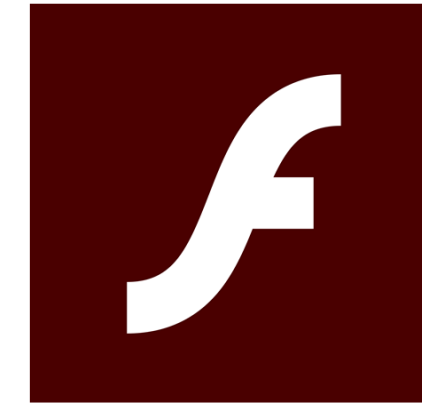


- Virtual memory
- Separate memory space of different processes



Web Browsers (2000s)

- Every functionality in one program
- Tons of rich contents
 - Flash, ActiveX, Java Applets
 - run on browser with user permission
- Drive-by downloads
 - Click on a malicious link
 - Trigger vulnerability
 - Pwned

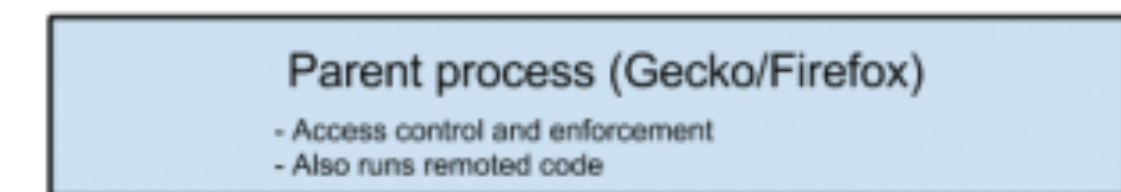


Solution: Kill them all!

New problems: Javascript engine, media codec, ...

The Rise of Sandboxing

- Untrusted code/data (i.e., downloaded JavaScript, PNGs, PDFs, etc) should live in a process with almost zero permissions.
- Spawn "privileged" parent process.
- Spawn "sandboxed" child processes.
- When a child needs to perform a privileged action, it asks the parent.
- need one set of vulnerabilities to exploit sandboxed process
- need another set of vulnerabilities to "break out" of the sandbox.



Significantly
raises the bar

Filesystem Isolation: chroot

- First appeared in Unix in 1979, BSD shortly afterwards.
- changes the meaning of "/" for a process (and its children)
- `chroot("/tmp/jail")` will disallow processes from getting out of the jail
- used to be the de-facto sandboxing utility
- no syscall filtering or other isolation

What does chroot do?

- `chroot("/tmp/jail")` has two effects:
 - For this process, change the meaning of "/" to mean "/tmp/jail".
 - For this process, change "/tmp/jail/.." to go to "/tmp/jail"
- `chroot("/tmp/jail")` does NOT:
 - Close resources that reside outside of the jail.
 - `cd (chdir())` into the jail.
 - Do anything else!
- What can go wrong?

chroot pitfalls: forgetting to chdir("/")

- You will not be in the jail
- You will be in an “unknown location”
- **getcwd** will not work.
- None of the regular UNIX utilities will work. (no access to /bin/bash or the library it uses.)
- **busybox** can help

```
$ busybox
BusyBox v1.18.0 (2010-12-01 19:10:28 CET) multi-call binary.
Copyright (C) 1998-2009 Erik Andersen, Rob Landley, Denys Vlasenko
and others. Licensed under GPLv2.
See source distribution for full notice.

Usage: busybox [function] [arguments]...
or: busybox --list[(-full)]
or: function [arguments]...

    BusyBox is a multi-call binary that combines many common Unix
    utilities into a single executable. Most people will create a
    link to busybox for each function they wish to use and BusyBox
    will act like whatever it was invoked as.

Currently defined functions:
[, [[, acpid, add-shell, addgroup, adduser, adjtimex, ar, arp, arping,
awk, base64, basename, bbconfig, beep, blkid, blockdev, bootchartd,
brctl, bunzip2, bzip2, cal, cat, catv, chat, chatr, chgrp,
chmod, chown, chpasswd, chpst, chroot, chrt, chvt, cksu, clear, cmp,
comm, conspy, cp, cpio, crond, crontab, cryptpw, ctttyhack, cut, date,
dc, dd, deallocvt, delgroup, deluser, depmod, devfsd, devmem, df,
dhcprelay, diff, dirname, dmesg, dnsd, dnsdomainname, dos2unix, dpkg,
dpkg-deb, du, dumpkmap, dumpleases, echo, ed, egrep, eject, env,
envdir, envuidgid, ether-wake, expand, expr, fakeidentd, false, fbset,
fbsplash, fdflush, fdformat, fdisk, fgconsole, fgrep, find, findfs,
flash_eraseall, flash_lock, flash_unlock, flashcp, flock, fold, free,
freeramdisk, fsck, fsck.minix, fsync, ftpd, ftpget, ftpput, fuser,
getopt, getty, grep, gunzip, gzip, halt, hd, hdparm, head, hexdump,
hostid, hostname, httpd, hush, hwclock, id, ifconfig, ifdown,
ifenslave, ifplugd, ifup, inetd, init, inotifyd, insmod, install,
ionice, iostat, ip, ipaddr, ipcalc, ipcrm, ipcs, iplink, iproute,
iprule, iptunnel, kbd_mode, kill, killall, killall5, klogd, last,
length, less, linux32, linux64, linuxrc, ln, loadfont, loadkmap,
logger, login, logname, logread, losetup, lpd, lpq, lpr, ls, lsattr,
lsmod, lspci, lsusb, lzcat, lzma, lzop, lzopcat, makedevs, makemime,
man, md5sum, mdev, mesg, microcom, mkdir, mkdosfs, mke2fs, mkfifo,
mkfs.ext2, mkfs.minix, mkfs.reiser, mkfs.vfat, mknod, mkpasswd, mkswap,
mktemp, modinfo, modprobe, more, mount, mountpoint, mpstat, msh, mt,
mv, nameif, nanddump, nandwrite, nbd-client, nc, netstat, nice, nmeter,
nohup, nslookup, ntpd, od, openvt, passwd, patch, pgrep, pidof, ping,
ping6, pipe_progress, pivot_root, pkill, pmap, popmaildir, poweroff,
powertop, printenv, printf, ps, pscan, pwd, raidautorun, rdate, rdev,
readahead, readlink, readprofile, realpath, reboot, reformime,
remove-shell, renice, reset, resize, rev, rfc2813, rm, rmdir, rmmount,
route, rpm, rpm2cpio, rtcwake, run-parts, runlevel, runsv, runsvdir,
rx, script, scriptreplay, sed, sendmail, seq, setarch, setconsole,
setfont, setkeycodes, setlogcons, setsid, setuidgid, sh, shasum,
sha256sum, sha512sum, showkey, slattach, sleep, smemcap, softlimit,
sort, split, start-stop-daemon, stat, strings, stty, su, sulogin, sum,
sv, svlogd, swapoff, swapon, switch_root, sync, sysctl, syslogd, tac,
tail, tar, taskset, tcpdump, tee, telnet, telnetd, test, tftp, tftpd,
time, timeout, top, touch, tr, traceroute, traceroute6, true, tty,
ttysize, tuncctl, tune2fs, ubiattach, ubidetach, udhcpc, udhcpd, udpsvd,
umount, uname, uncompress, unexpand, uniq, unix2dos, unlzma, unlzop,
unxz, unzip, uptime, usleep, uudecode, uuencode, vconfig, vi, vlock,
volname, wall, watch, watchdog, wc, wget, which, who, whoami, xargs,
xz, xzcat, yes, zcat, zcip
```

```
$ _
```


chroot pitfalls: previously open resources

- Previously-open resources remain unaffected.
- How is this useful?
- Similar to **open** and **execve**, Linux has **openat** and **execveat**:

```
int open(char *pathname, int flags);  
int openat(int dirfd, char *pathname, int flags);  
int execve(char *pathname, char **argv, char **envp);  
int execveat(int dirfd, char *pathname, char **argv, char **envp, int flags);
```

- **dirfd** can be a file descriptor representing any **open()**ed directory
- Many other system calls also have "at" variants!

chroot pitfalls: forgetfulness

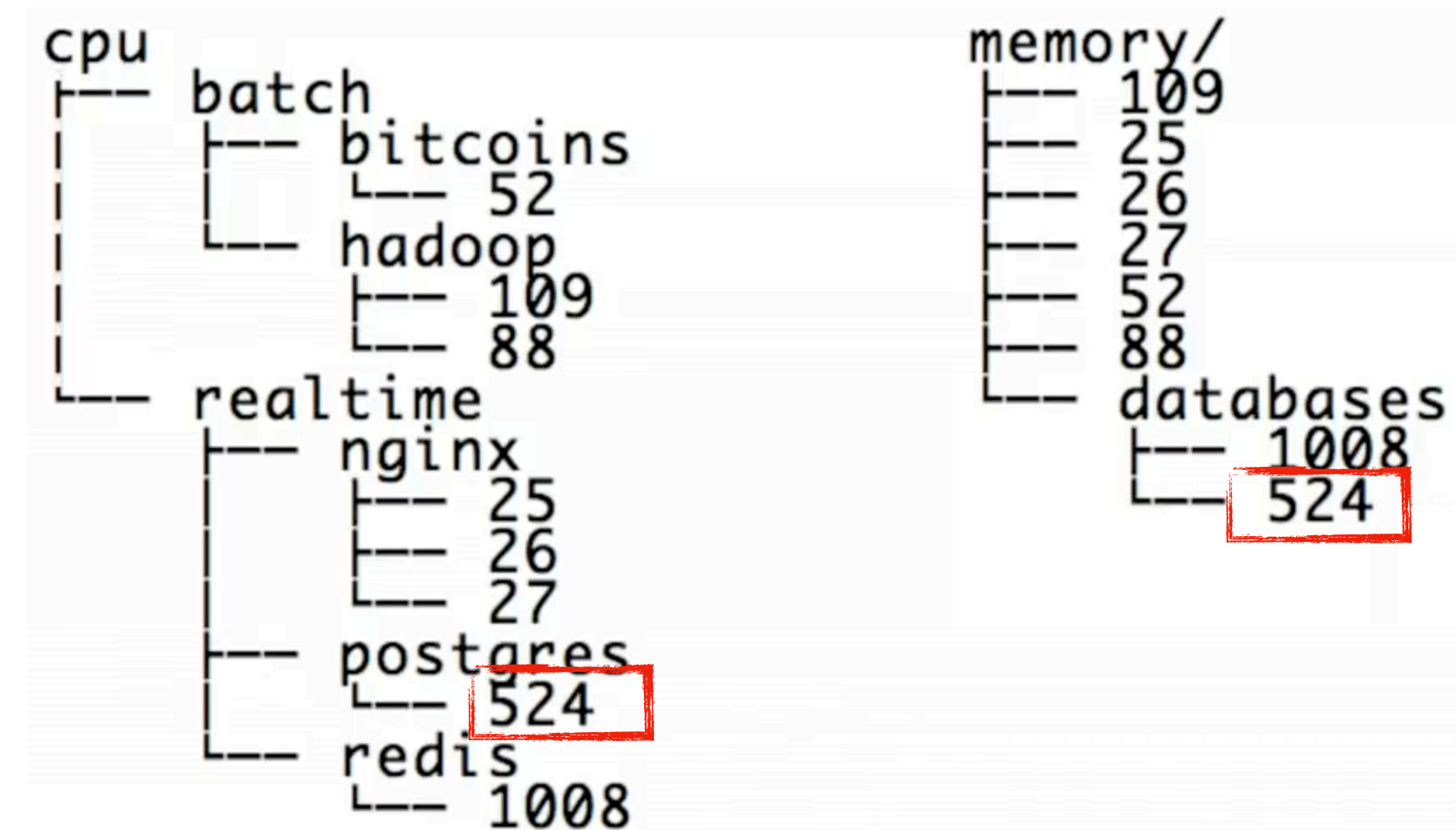
- One chroot at a time.
- What happens if you chroot again?
- It will be overwritten!

Is chroot safe?

- Generally, a user with an effective ID of 0 (i.e., a process run as root or SUIDed to root) can *always* break out of a chroot, unless the chroot syscall is blocked!
- Missing other forms of isolation:
 - PID
 - network
 - IPC
- Replacements:
 - cgroups
 - namespaces
 - seccomp

cgroups

- Limits what you can do
- Each subsystem (e.g. CPU, memory, block I/O) has an independent tree
- Each process is in a node in each tree
- Each hierarchy starts with 1 node (root)
- Each node = group of processes
- New processes start in their parent's group
- Can be moved around



memory cgroup

- Accounting

- Keep track of pages used by each group
 - file (disk page)
 - anonymous (stack, heap, mmap)
- Each page is “charged” to a group
- Pages can be shared across multiple groups
- One group “pays” for it
- Adds some overhead

- Limits

- Can be set for different types of memory: RAM, swap, total, ...
- **Hard limits:** You go above, a random process dies (More sophisticated options available)
- **Soft limits:** Candidate for reclaim under memory pressure

CPU cgroup

- Keep track of user/system CPU time
- Keep track of usage per CPU
- Set weights

CPUSETS cgroup

- Pin groups to specific CPU(s)
- Reserve CPUs for specific apps
- Avoid processes bouncing between CPUs

BLKIO cgroup

- Keeps track of I/Os for each group per device
 - read/write, sync/async
- Set limits for each group per device
 - read/write, ops/bytes
- Relative weights for each group

Devices cgroup

- What the group can do on device nodes
- read/write/mknod

freezer cgroup

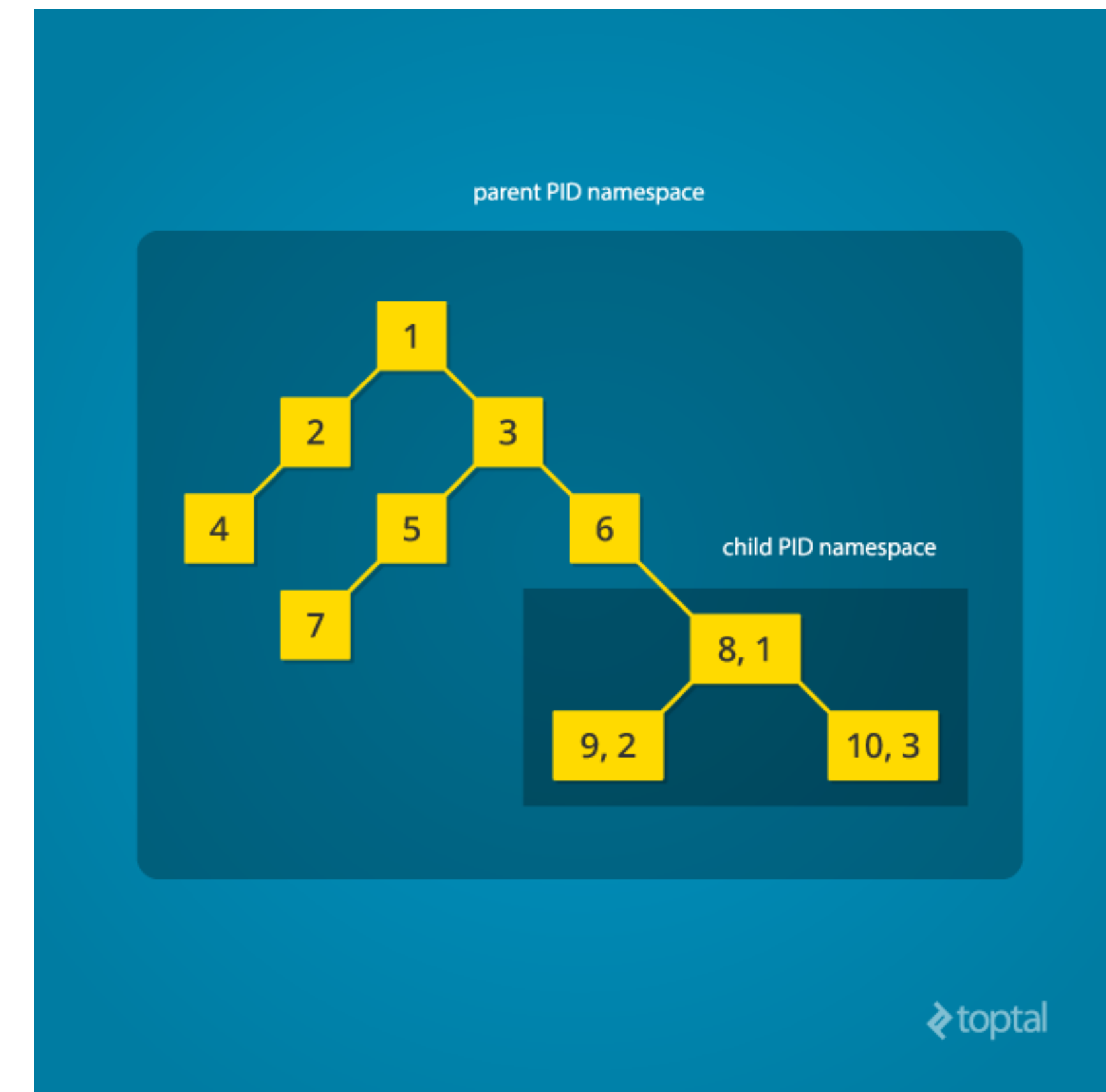
- Freeze/thaw a group of processes
- Can't be detected by the process
- doesn't impede ptrace/debugging
- Specific usecases
 - Cluster batch scheduling
 - Process migration

namespaces

- Limiting what you can view
- Provide processes with their own system view
- Multiple namespaces:
 - pid
 - net
 - mnt
 - uts
 - ipc
 - user
- Each process is in one namespace of each type

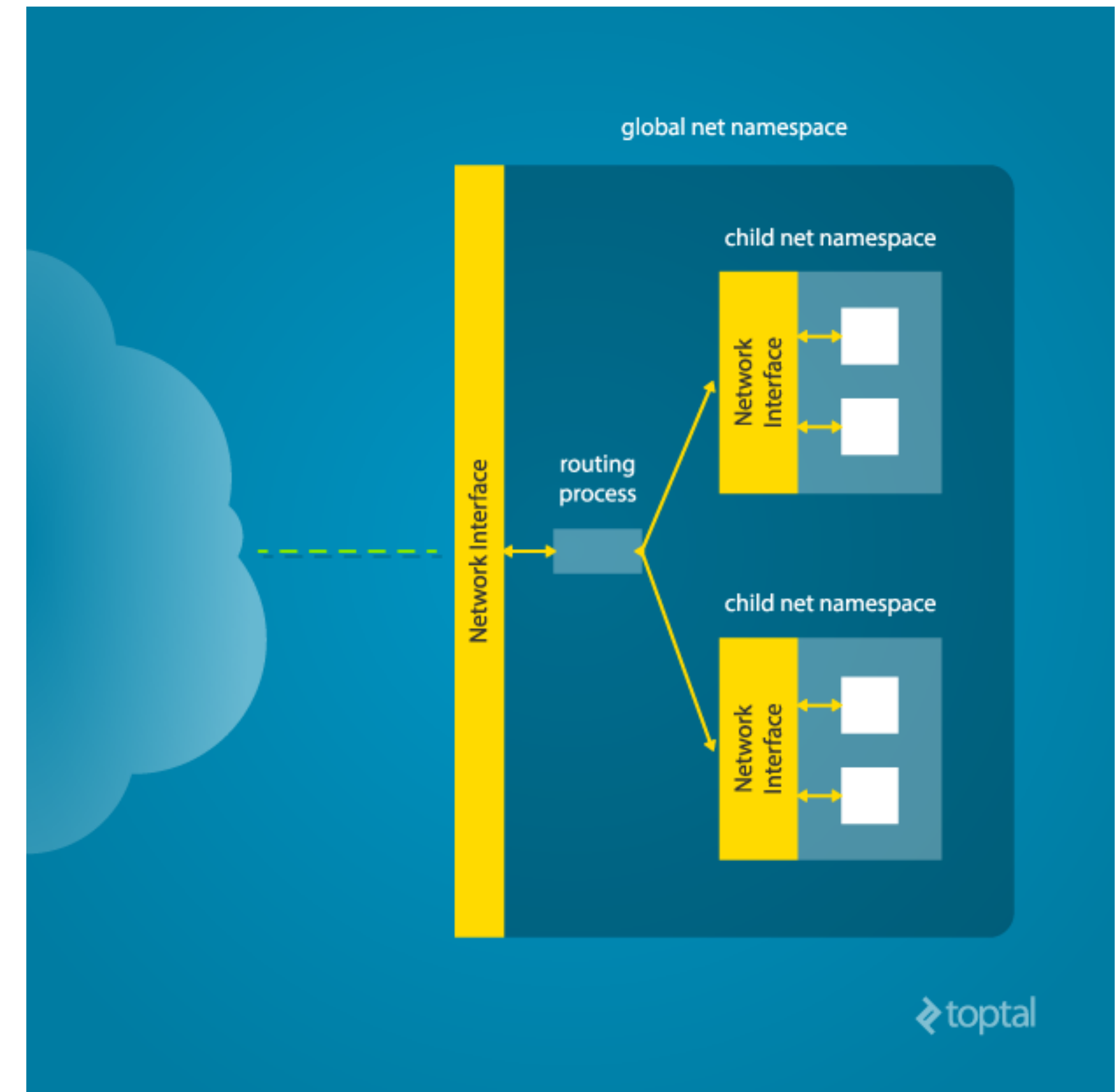
pid namespace

- Processes within a PID namespace only see processes in the same PID namespace
- Each PID namespace has its own numbering (starting with 1)
- If PID 1 goes away, the whole namespace is killed
- Supports nesting
- A process can have multiple PIDs (one/namespace)



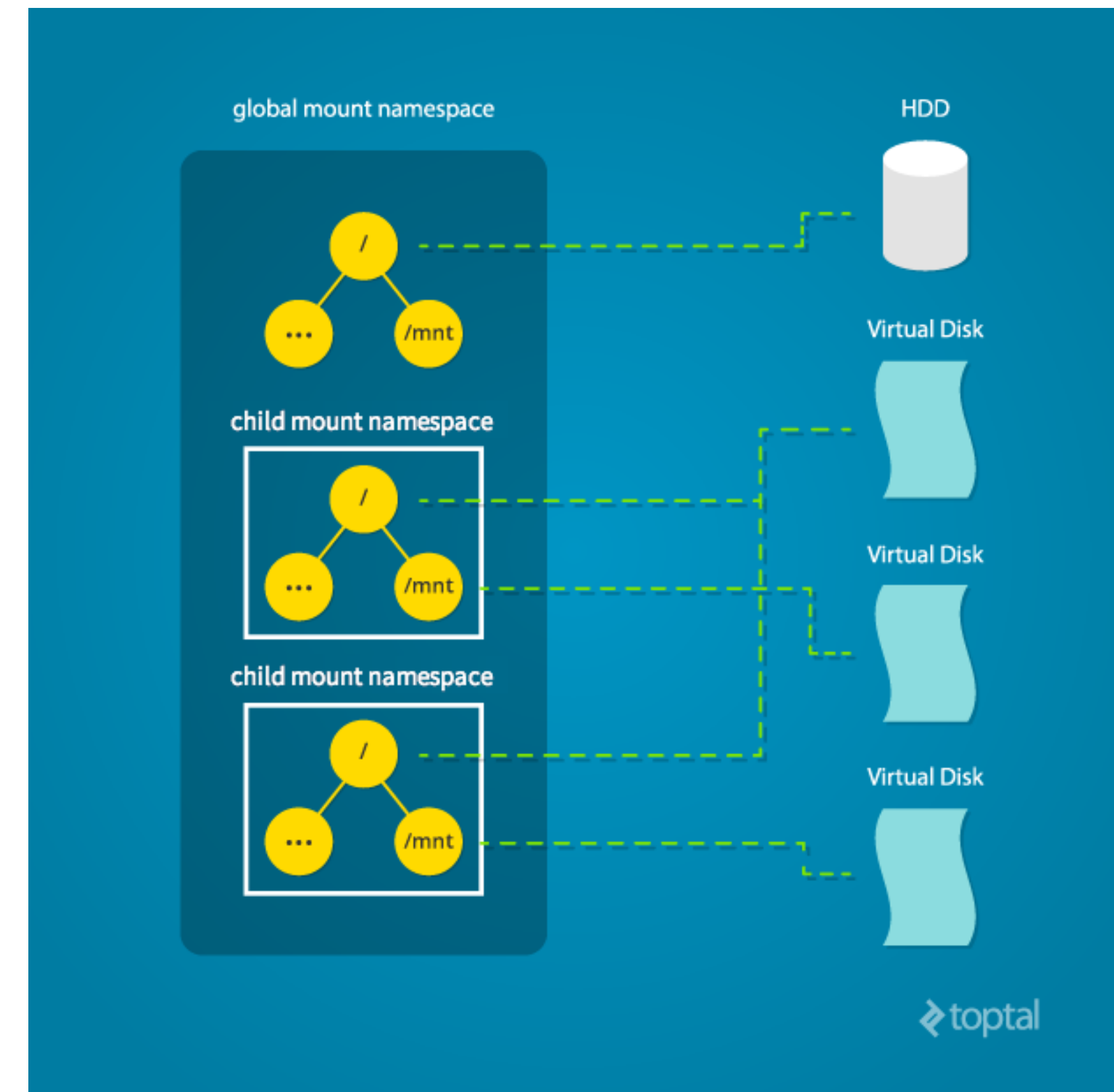
net namespace

- Processes within a given network namespace get their own private network stack
 - network interface (including IO)
 - routing tables
 - iptable rules
 - sockets
- You can move a network interface across net namespaces



mnt namespace

- Each process can have their own root fs (hello chroot!)
- Each process can have their “private” mount
- A child process gets a copy of parent process current mounts but diverges afterward
- Mounts can be shared
- No easy way to pass along



user namespace

- Map uids
- **The "Root" Illusion:** uid in containers becomes irrelevant
 - Just use UID 0
- **Privilege Isolation:** Capabilities (like CAP_SYS_ADMIN) granted inside a user namespace are valid only within that namespace and do not apply to the host.
- **Mitigating Container Escapes:** If an attacker breaks out of a container where user namespaces are active, they find themselves as an unprivileged user on the host rather than a root user.
- **Reduced Attack Surface:** Many kernel vulnerabilities that require root privileges cannot be exploited from within a user namespace because the "root" user there lacks real host-level permissions.

Other Namespaces

- IPC Namespace: Isolates IPC objects and POSIX message queues
- uts Namespace: Isolates hostname and NIS domain name configuration

namespace Manipulation

- Created with clone() system call.
- You can view them at /proc/<pid>/ns
- (Typically) When the last process of a namespace exits, it is destroyed
- You can enter a namespace through setns()

Seccomp: Syscall Filtering

- Heavily restrict permitted system calls through the use a kernel-level sandboxing mechanism
- seccomp allows developers to write complex rules to:
 - allow certain system calls
 - disallow certain system calls
 - filter allowed and disallowed system calls based on argument variables
- seccomp rules are inherited by children.
- Applies even to root.
- Rules can get quite complex.
- Used by docker, chrome, firefox, ...

Simple Example

```
scmp_filter_ctx ctx;  
  
ctx = seccomp_init(SCM_ACT_ALLOW);  
  
seccomp_rule_add(ctx, SCMP_ACT_KILL, SCMP_SYS(execve), 0);  
  
seccomp_load(ctx);
```

How does seccomp work?

- Uses the kernel functionality eBPF.
- **e**xtended **B**erkeley **P**acket **F**ilters are programs that run in an in-kernel, "provably-safe" virtual machine.
- Can instrument kernel functionality in very general ways.
 - originally used to filter network traffic (iptables)
 - i.e., used to implement system-wide syscall tracing
- Used with seccomp() to apply syscall filters to processes.

How to Escape seccomp?

- Generally, to do anything useful, a sandboxed process needs to be able to communicate with the privileged process.
- Normally, this means allowing the sandboxed process to use **some** system calls. This opens up some attack vectors:
 - permissive policies
 - syscall confusion
 - kernel vulnerabilities in the syscall handlers

seccomp Pitfalls: Permissive Policies

- System calls are complex, and numerous.
- Developers might avoid breaking functionality by erring on the side of permissiveness.
- **Well-known example:**
 - depending on system configuration, allowing the `ptrace()` system call could let a sandboxed process to "puppet" a non-sandboxed process.
- **Some less well-known effects:**
 - `sendmsg()` can transfer file descriptors between processes
 - `prctl()` has bizarre possible effects
 - `process_vm_writev()` allows direct access to other process' memory

seccomp Pitfalls: Syscall Confusion

- Many 64-bit architectures are backwards compatible with their 32-bit ancestors:

amd64	/	x86_64	x86
		aarch64	arm
		mips64	mips
		powerpc64	ppc
		sparc64	sparc

- On some systems (including amd64), you can switch between 32-bit mode and 64-bit mode in the same process, so the kernel must be ready for either.
- Interestingly, system call numbers differ between architectures, including 32-bit and 64-bit variants of the same architecture!
- Policies that allow both 32-bit and 64-bit system calls can fail to properly sandbox one or the other mode.
- Example: `exit()`
 - syscall 60 (mov rax, 60; syscall) on amd64
 - syscall 1 (mov eax, 1; int 0x80) on x86.
- Seccomp policy has different entries for 32 and 64bit syscalls

seccomp Pitfalls: Kernel Vulnerability

- If the seccomp sandbox is correctly configured, the attacker can't do anything useful.
- But they can still interact with the system calls that are allowed! This allows the attacker to try to trigger vulnerabilities in the kernel.
- **Powerful!** Over 30 Chrome sandbox escapes in 2019 alone:
<https://github.com/allpaca/chrome-sbx-db>

seccomp vs. data exfiltration

- seccomp works well when you want to stop code execution.
- **What if your goal is data exfiltration?**
- Even if you can't directly communicate with the outside world, often you can send "smoke signals":
 - Runtime of a process (see sleep(x) system call) can convey a lot of data.
 - Clean termination or a crash? This can convey one bit.
 - Return value of a program (exit(x)) can convey one byte.
- Real-world example: attackers use DNS queries to bypass network egress filters.
- **As long as you can communicate 1 bit, you can repeat the attack to get more and more bits!**

lxc: Linux Container

- Combines chroot with Namespaces and Cgroups.
- Has its own process space, network interface
- Can run binaries as root, install packages, change routing and ip tables
- Uses host kernel
- A light-weight VM: each process in its own bubble